Experiment 7

Student Name: Shivanshu Ranjan **UID: 23BCS10193**
Branch: CSE **Section/Group: KRG 3-B**
Semester: 6th **Date of Performance:03/04/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design and analyze a scalable **Ride-Sharing Service (like Ola/Uber)** that allows users to book rides, track drivers in real-time, and make payments while ensuring high availability, low latency, and efficient driver-rider matching using distributed system architecture.

2. Objective:

- To identify functional and non-functional requirements of a ride-sharing system
- To design a High Level Architecture (HLD) for ride booking systems
- To understand real-time location tracking and driver matching
- To design microservices for ride booking, payments, and notifications
- To understand scalability challenges in ride-sharing apps
- To study how systems like Uber scale to millions of users

3. Tools:

- Draw.io / Excalidraw – System design diagrams
- MySQL / PostgreSQL – User & Ride data storage
- Redis – Caching & real-time matching
- Kafka / Queue – Event processing (optional)
- Load Balancer – Traffic distribution
- Google Maps API – Location services
- WebSockets – Real-time tracking
- Microservices Architecture

4. System Requirements:

A. Functional Requirements:-

1) User should be able to register and login
2) User should be able to get fare estimation (pickup → drop)
3) User should be able to request a ride
4) User should be able to choose ride types (car, bike, etc.)
5) Drivers should be able to accept/reject ride requests
6) System should match riders with nearby drivers
7) Real-time tracking of driver location

8) Users and drivers should be able to rate each other
9) User should be able to make payments

B. Non-Functional Requirements:-

1) **Scalability**:
    a. Support millions of users and drivers (~1.5M+)
2) **Availability**:
    a. System should always be available for booking
3) **Consistency**:
    a. A driver must not be assigned to multiple riders (Strong Consistency)
    b. User data can follow eventual consistency
4) **Latency**:
    a. Driver matching should happen within < 1 minute
5) **Reliability**:
    a. Ride requests should not be lost

5. High Level Design (HLD):

System follows Microservices Architecture

Main Components:

1) API Gateway
    a. Authentication
    b. Authorization
    c. Routing
    d. Rate limiting
2) User Service
    a. Registration & login
    b. User DB

3) Ride Service
    a. Fare calculation
    b. Ride creation
4) Driver Matching Service
    a. Finds nearest drivers (Geo-hash / location-based)
5) Location Service
    a. Real-time driver tracking
6) Notification Service
    a. Sends ride requests to drivers
7) Payment Service
    a. Handles payments
8) Rating Service
    a. Stores ratings

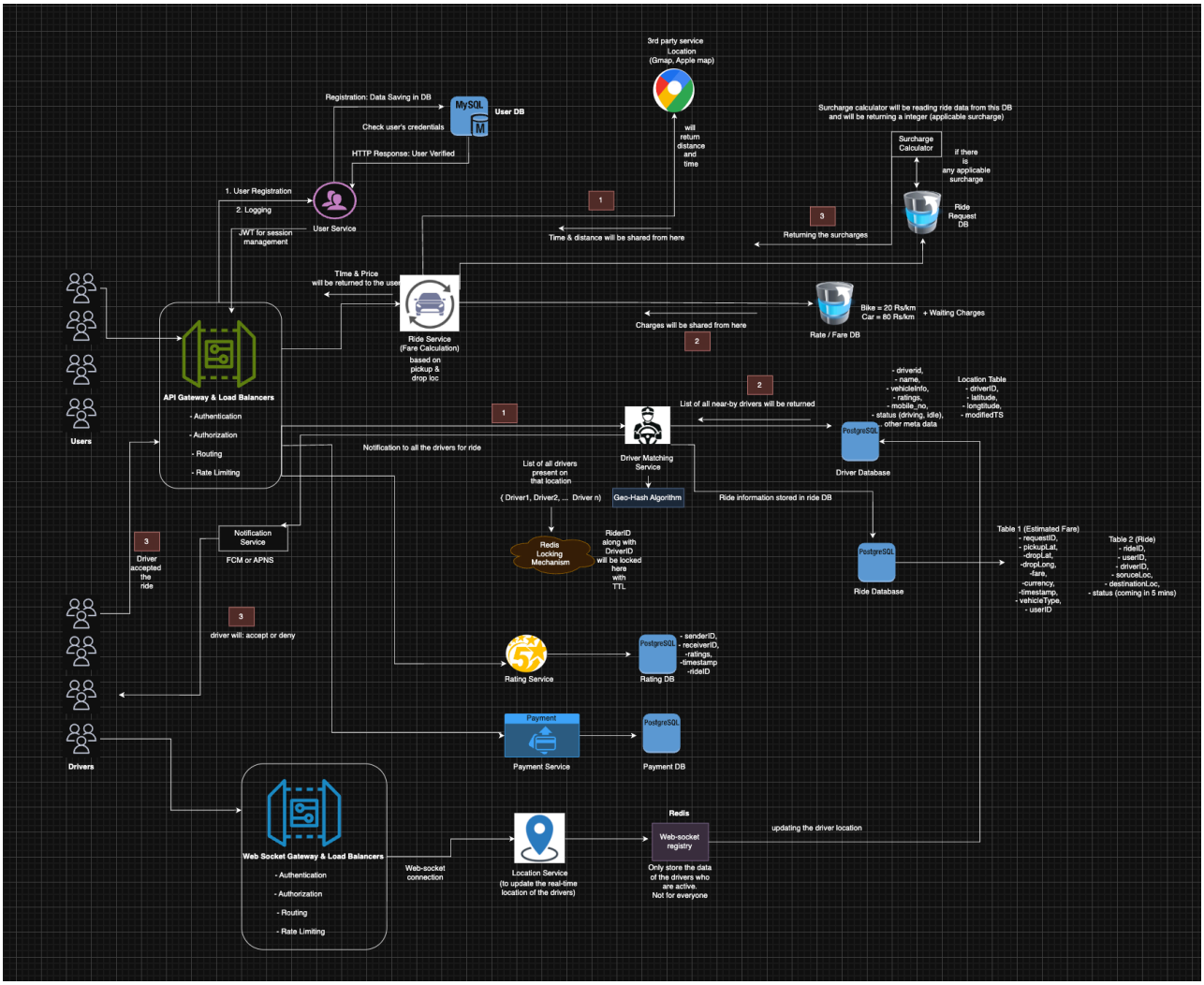
6. Ride Flow (Step-by-Step):

- User enters pickup & drop location
- System calculates fare
- User requests ride
- Request goes to Driver Matching Service
- Nearby drivers are notified
- Driver accepts ride
- Ride starts → location tracking enabled
- Ride ends → fare calculated
- Payment completed
- Rating submitted

7. Scalability Solution:

To support millions of users:

- Use Geo-hashing for nearby driver search
- Use Redis caching for fast lookup
- Use WebSockets for real-time updates
- Use Load Balancers to distribute traffic
- Use Microservices for scalability
- Use Separate databases (User, Ride, Driver, Payment)
- Use Queue (Kafka) for async processing
- Use CDN for maps & static content



8. Learning Outcomes (What I Have Learnt):

- **Learned how ride-sharing apps like Uber are designed**
- **Understood real-time system design concepts**
- **Learned driver matching using location-based services**
- **Understood microservices architecture**
- **Learned how to handle scalability and consistency issues**
- **Understood real-time tracking using WebSockets**
- Learned how distributed systems manage millions of users